PROJECT NOTES · INTERACTIVE SYSTEMS AND CAD

Kill Web and Autonomous Systems CAD Studies

A browser command game, three CAD assemblies with a shared web inspection viewer, and two interactive isometric anatomy figures.

Parth Patel · November 2026 · parth-patel.me

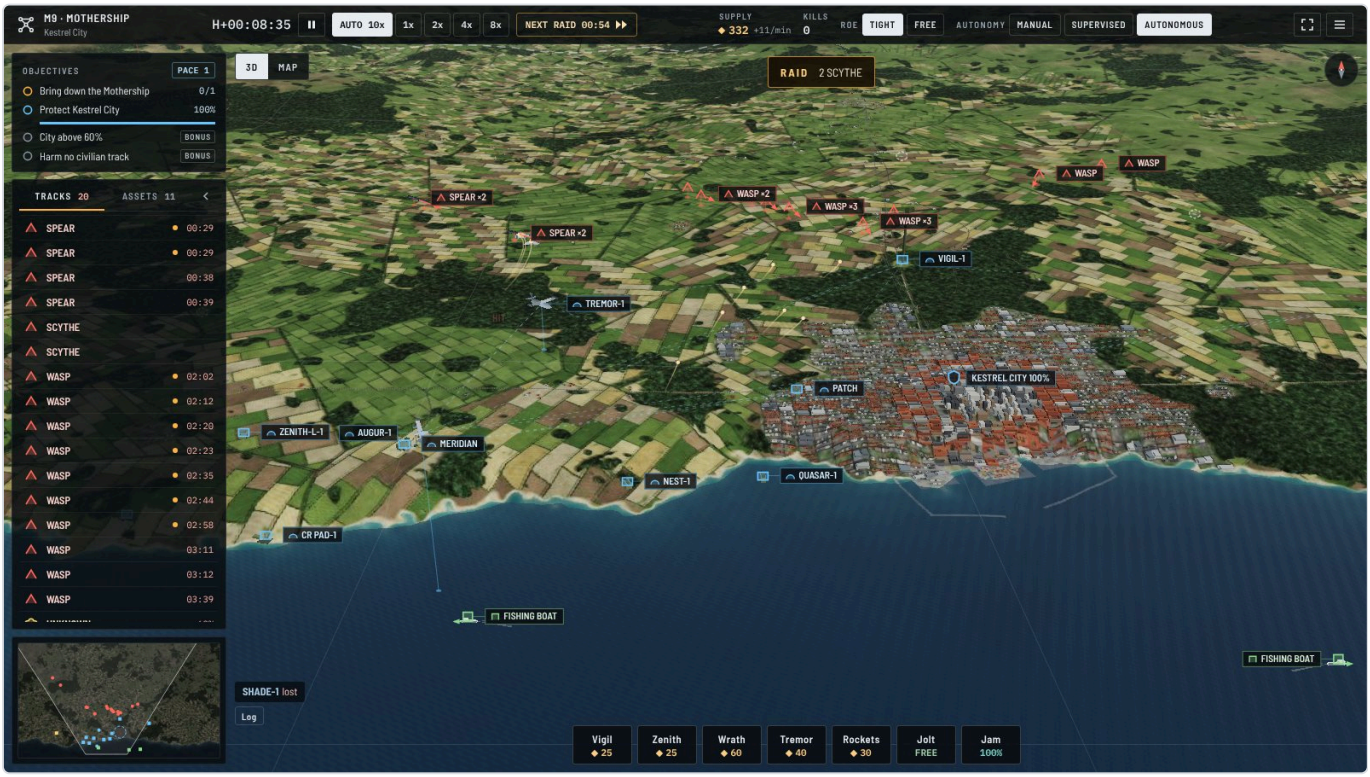


Figure 1. Mission 9 (Mothership) at Pace 1, 8 min 35 s in: 20 tracks on the board as a raid closes on Kestrel City, autonomy set to Autonomous. Objectives and the track list run down the left, the deployment dock along the bottom, the raid banner at the top.

Kill Web is a real-time command game that runs in any modern browser. The player finds every track, identifies it, then pairs the right shooter; nothing fires until a track is identified. These notes cover the game's design, architecture and balance testing, then three CAD assemblies built in Python and inspected in a custom three.js viewer, then two interactive isometric figures. All performance values in the game are fictional, and the published CAD renders show exterior shape only.

9 · 5 missions in 3 acts · pace levels	**6,212** lines of JavaScript in 10 ES modules, no framework, no build step	**0.25 s** fixed simulation step, shared by the browser and the Node test harness
25,182 generated buildings on nine 48 × 38 km maps	**92.7%** bot win rate over 450 scripted runs (9 missions × 5 paces × 10 seeds)	**278 · 0** unique CAD parts across 3 assemblies · interference clashes

1. Kill Web: the game

1.1 Concept and systems

The player commands Task Force Warden on the fictional Kestrel Coast against the fictional Vanta Combine. Its 15 unit types, from sensor towers to jet aircraft, carry fictional names and game values, loosely modeled on publicly shown systems. Sensors turn contacts into tracks, identification turns an unknown track into a named hostile or a neutral (shown as a confidence from 0 to 100%), and only then can the player, or the autonomy setting, pair a shooter. Civilian aircraft and fishing boats share the map, and harming one fails a bonus objective.

- **Autonomy and rules of engagement.** Manual (every shot approved), Supervised or Autonomous; rules of engagement Tight or Free.
- **Supply.** Deployments draw on supply that accrues per minute, and each dock button shows its cost: Vigil 25, Zenith 25, Wrath 60, Tremor 40, Rockets 30, Jolt free.
- **Tempo.** The clock runs at 10× through quiet stretches and drops to 3× while anything is in contact (Pace 1 values), so long quiet stretches pass quickly without hiding the fights.
- **Progression.** XP, credits and an 18-skill tree in three branches (Sense, Decide, Act) carry across the campaign. The whole arsenal is available from mission 1.

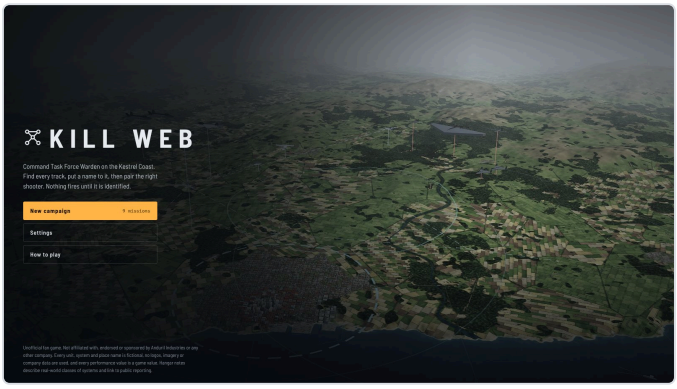


Figure 2. Title screen over the generated Kestrel City terrain.

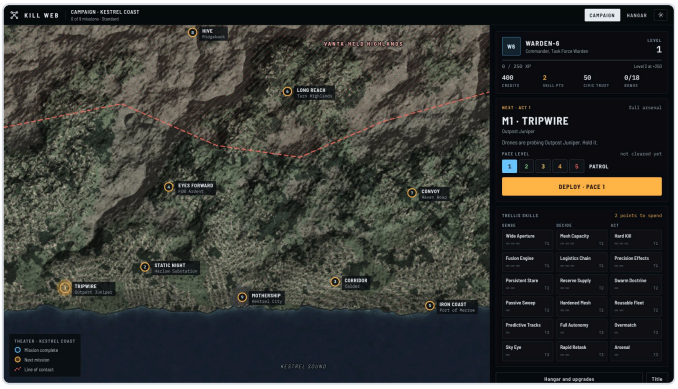


Figure 3. Campaign map: nine mission pins, commander record and skill tree.

Table 1. Campaign

#	Mission	Place	Act	Conditions	Objectives
1	Tripwire	Outpost Juniper	1	Day, training	4 (2 bonus)
2	Static Night	Harlow Substation	1	Night	5 (2 bonus)
3	Corridor	Calder	1	Day	5 (2 bonus)
4	Eyes Forward	FOB Ardent	2	Day	5 (2 bonus)
5	Iron Coast	Port of Merrow	2	Day	6 (2 bonus)
6	Long Reach	Tarn Highlands	2	Day	5 (2 bonus)
7	Convoy	Haven Road	3	Day, escort	3 (2 bonus)
8	Hive	Ridgeback	3	Day	5 (2 bonus)
9	Mothership	Kestrel City	3	Day, finale	4 (2 bonus)

1.2 Pace levels

Pace replaces unlock grinding as the difficulty axis. It scales the enemy faster than the player: at Overrun, raids arrive 1.75 times as often and fly 1.45 times as fast, while friendly deployment, supply and reloads speed up by 1.4. Changing pace on the briefing rebuilds the simulation with the same seed, so a replay at a higher pace is the same battle, faster. Rewards scale with pace.

Table 2. Pace multipliers (applied in sim.js)

Level	Enemy speed	Raid tempo	Deploy, supply, reload	Shot speed	Clock, quiet / contact	Reward
1 Patrol	1.00	1.00	1.00	1.00	10× / 3×	1.0×
2 Alert	1.10	1.15	1.10	1.10	11× / 3.25×	1.2×
3 Surge	1.20	1.32	1.20	1.20	12× / 3.5×	1.4×
4 Blitz	1.32	1.52	1.30	1.30	13× / 3.75×	1.6×
5 Overrun	1.45	1.75	1.40	1.40	14× / 4×	1.8×

1.3 Architecture

The game is ten ES modules loaded directly by the browser. The simulation never touches the DOM or WebGL, which is what lets the same `sim.js` run under Node for testing. It advances in fixed 0.25 s steps, up to 24 per rendered frame, and the renderer interpolates every position between the last two steps, so motion stays smooth at any clock speed.

Table 3. Modules

Module	Lines	Responsibility
sim.js	2,038	Deterministic simulation: entities, sensors, tracking and identification, engagements, raids, objectives
mission.js	1,100	Mission screen: frame loop, automatic tempo, HUD, deployment dock, input
world.js	895	three.js scene: terrain, instanced buildings, stylized unit models, effects, camera
data.js	612	Missions, unit and weapon tables, products, skills, pace table
overlay.js	532	2D canvas over the 3D view: labels that avoid each other and the side panels, range rings, kill callouts
menus.js	492	Campaign map, hangar, debrief
app.js	305	Boot, profiles, rewards, screen flow
audio.js	105	Synthesized sound: every cue is built from oscillators and filtered noise, nothing is downloaded
store.js	72	Save storage in the browser (localStorage)
blueprint.js	61	Hangar line drawings from the same stylized models the battlefield uses



Figure 4. Mission 9 terrain at a closer zoom: field patterns, forests, coastline and friendly assets, all generated from the simulation's own terrain functions.

Terrain. Each mission loads a 640 × 507 height mesh under a 4,096 × 3,243 ground texture covering 48 × 38 km, plus a 400 × 304 far ring out to 108 × 82 km for the horizon. The textures, height maps and building layouts are generated offline in Python (NumPy, Pillow) from the simulation's own height, shore and road functions, dumped through Node, so the 3D view and the simulation agree on every coast and road. Buildings are drawn as instanced boxes, 1,491 to 6,377 per mission. No satellite imagery or third-party art is used.

Delivery. The site is static: 78 files, 36.8 MB, served by GitHub Pages at `parth-patel.me/killweb/`. three.js 0.160 loads from the jsDelivr CDN and progress saves to the player's browser. It plays in landscape on desktop, tablet and phone.

1.4 Balance testing

A scripted commander plays every mission headless in Node, using only the public command methods the UI calls, with a fixed loadout per mission. One sweep runs 10 fixed seeds of each mission at each pace, 450 reproducible runs in about 4 minutes on one core, and logs outcome, mission time, kills, protected-site health, civilian harm and losses for every run.

Table 4. Latest sweep (scripted commander, Supervised autonomy)

Pace	Runs	Wins	Win rate	Median winning time
1 Patrol	90	83	92.2%	36 min
2 Alert	90	79	87.8%	32 min
3 Surge	90	83	92.2%	28 min
4 Blitz	90	83	92.2%	25 min
5 Overrun	90	89	98.9%	22 min
All	**450**	**417**	**92.7%**	

Missions 1 to 5 were won in all 250 of their runs; every loss came from missions 6 to 9. The bot wins most often at Overrun because it reacts instantly, so faster deployment helps it more than faster raids hurt it. A human's limit is attention, which is

exactly what pace compresses: median mission time falls from 36 to 22 game minutes. The harness also caught design faults before players did:

- **Convoy got easier at higher pace.** Raids arrived before the convoy had left. Departure time and convoy speed now scale with raid tempo.
- **The finale regressed after two aircraft roles merged into one.** The single aircraft spent its whole magazine in about 2 s on the first targets in range, then was lost. Tasking now ranks targets by value, the airframe is tougher, and a lost aircraft returns after a 120 s cooldown.

2. CAD studies

2.1 Scope

Three assemblies were modeled to practice large-assembly structure, part numbering and automated checking, and to build one reusable browser viewer for all of them. Two are representative models of publicly shown aircraft, built only from public photographs and published overall dimensions; they are not manufacturer data, and their names here are fictional. The third, Shade-X, is an original design for a home-buildable, quiet ISR helicopter with a 3D-printed and cut-plate frame, belt drive and electric tail around an off-the-shelf rotor.

Table 5. Assemblies

Assembly	Basis	Unique parts	Instances	Subsystems	Clashes	Floating
Zenith-6 tube-launched UAS	Representative, public imagery	60	150	6	0	0
Wrath collaborative combat aircraft	Representative, public imagery	125	130	7	0	0
Shade-X quiet ISR helicopter	Original home-built design	93	142	8	0	0
Total		**278**	**422**	**21**	**0**	**0**

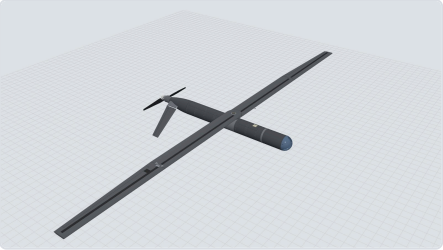


Figure 5. Zenith-6 tube-launched UAS, flight configuration.



Figure 6. Wrath CCA, clean configuration.



Figure 7. Shade-X concept, original design.

2.2 Modeling pipeline

Every part is a parametric B-rep solid written in Python with CadQuery 2.8 on Open CASCADE 7.9, numbered by subsystem (for example 100 structure, 200 wing, 400 propulsion) and placed as one or more instances. The pipeline tessellates each unique part once, encodes the meshes as typed arrays and writes them, with the assembly tree and parts list as JSON, into one self-contained HTML page per assembly: 1.8 MB for Zenith-6, 2.2 MB for Wrath and 4.3 MB for Shade-X, with no server or plugin.

2.3 Interference and support checks

The two fixed-wing models were run through a two-stage automated check:

- **Mesh screen.** All instance pairs are intersected as meshes tessellated at 0.05 mm deflection.

- **Exact confirmation.** Pairs whose mesh overlap exceeds 0.2 mm³ are re-tested with exact B-rep booleans, both fuzzy (tolerance 1e-4) and plain.
- **Clash rule.** A clash is more than 1.0 mm³ of overlap in both methods, or more than 1.0 mm³ in one with mesh penetration deeper than 0.15 mm. A gap under 0.25 mm counts as contact.
- **Support.** Every part must touch at least one other part, so nothing floats.

Both report 0 clashes, 0 floating parts and 0 check timeouts or failures, the tube-launched model in both its flight and stowed configurations. The helicopter model also reports 0 clashes and 0 floating parts.

2.4 Inspection viewer

The viewer is a three.js 0.160 page with one instanced mesh per part number and an edge overlay. It offers orbit, pan and zoom, four standard views, an explode slider, a section plane at the centerline with live clipping, coloring by material or by subsystem, and click-to-select with a part card. The tube-launched model adds flight and stowed configurations with an animated wing fold.

> **What is published.** The public renders show exterior shape only, in clean configurations. Parts lists, masses, materials, internal layouts and the interactive viewers stay private, and the Wrath render omits its underwing stores and pylons.

3. Isometric anatomy figures

Two interactive figures explain how a field-portable helicopter kit and a tube-launch test deck come apart. Each is a single HTML page drawn as SVG in a fixed isometric projection, with four-tone face shading and back-to-front painter's ordering; in the helicopter figure, WebGL layers draw the rotor disc and rotor wash in the same projection. Hovering or the arrow keys explode the assembly, and buttons run spin-up, battery swap, load-and-launch and motor sequences. The figures pause when off-screen and stay still under reduced-motion settings.

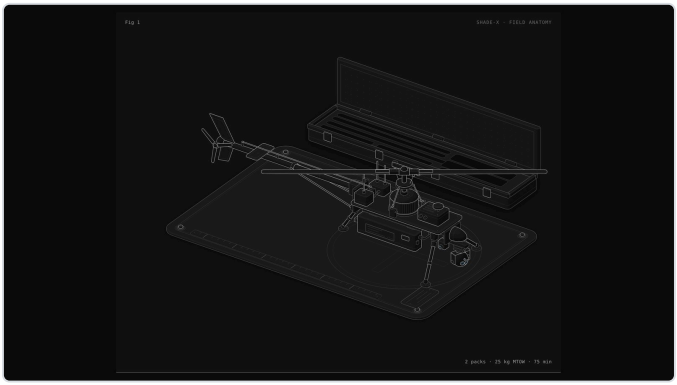


Figure 8. Shade-X field kit: 150 moving part groups in 10 assemblies, 2 WebGL layers.

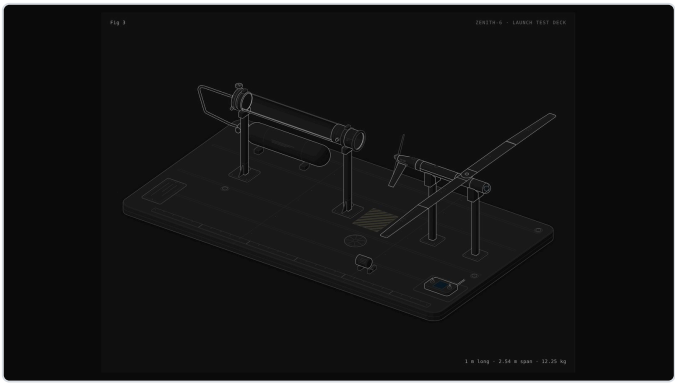


Figure 9. Zenith-6 launch test deck: 151 part groups, 13 explode steps.

4. Legal, sourcing and safety notes

- **Unofficial.** This is a personal, non-commercial portfolio project. It is not affiliated with, endorsed or sponsored by Anduril Industries, Inc. or any other company.
- **Fictional names.** Every unit, system and place name in the game, and the names of the CAD studies and figures, are fictional. No company logos, wordmark styling, photographs, documents or data are used.
- **Original assets.** All game terrain, buildings, key art, unit models and sound are generated by code in this project. Third-party software: three.js (MIT License); fonts IBM Plex and Barlow Semi Condensed (SIL Open Font License 1.1) via Google Fonts.

- **Fictional values.** The Vanta Combine, the Kestrel Coast, the cast and every performance number in the game are fictional. Hangar notes describe real-world classes of systems and link to public reporting.
- **CAD.** Two models are representative studies of publicly shown aircraft, approximated from public photographs and published overall dimensions. Only exterior renders are published; no internal design, material or performance data is.
- **Requests.** Questions or takedown requests are welcome through parth-patel.me/contact.html and will be acted on promptly.

5. Sources and credits

- **Public reporting.** Hangar notes in the game link to articles from The War Zone, The Aviationist, TechCrunch, Microwave Journal, ExecutiveBiz, Orange County Business Journal, Seedtable and Designation-Systems.net.
- **three.js** 0.160, MIT License. `github.com/mrdoob/three.js`
- **Fonts.** IBM Plex Sans, IBM Plex Mono and Barlow Semi Condensed, SIL Open Font License 1.1.

Tools. JavaScript (ES modules), three.js 0.160, WebGL, WebAudio, Python 3 (NumPy, Pillow, CadQuery 2.8 on Open CASCADE 7.9), Node.js, and Playwright with headless Chromium for screenshots and tests. Developed in AI-assisted sessions with Claude (Anthropic).